INNOMATICS®
RESEARCH LABS

INNOVATION. AUTOMATION. ANALYTICS

PROJECT ON

E-COMMERCE REFRIGERATOR DATA ANALYSIS USING WEB SCRAPING & PYTHON

About me

- M. Lakshmikar Master in MBA
- ID: 81010819725 Batch: 448

INNOMATICS
RESEARCH LABS

Objective of the Project

To design and implement an end-to-end web scraping and data analysis pipeline that extracts refrigerator product data from an e-commerce platform, transforms unstructured HTML and inconsistent textual attributes using Python and Regular Expressions, and performs exploratory data analysis to generate structured insights for comparative market analysis.

Web Scraping – Approach

- Tools used:
  - requests
  - BeautifulSoup
- Extracted attributes:
  - Brand
  - Price
  - Doors of the product
  - Water Capacity
  - Stars of the Product
  - Built-in Stabilizers
  - Compressor Models
  - Warranty of the Product
  - Compressor Warranty
  - Discounts
  - Ratings

```python
import re
import requests
from bs4 import BeautifulSoup
import numpy as np
import pandas as pd
```

```python
BRAND = []
PRICE = []
DOORS = []
WATER_CAPACITY = []
STARS = []
WARRANTY = []
STABILIZER = []
COMPRESSOR_MODEL = []
COMPRESSOR_WARRANTY = []
PRICE_OFF = []
RATINGS = []
```

```python
PRODUCTS_PER_PAGE = 24

for page in range(1, 31):
    url = f"https://www.flipkart.com/search?q=refrigerator&otracker=search&otracker1=search&marke
    print(url)

    r = requests.get(url)
    soup = BeautifulSoup(r.text, "html.parser")

    # PRODUCT NAMES
    data = soup.find_all('div', class_="RG5Slk")

    assert len(data) >= PRODUCTS_PER_PAGE, f"Only {len(data)} products found on page {page}"

    data = data[:PRODUCTS_PER_PAGE]
    for i in data:
        BRAND.append(i.text.split()[0])

    # WATER_CAPACITY
    for i in data:
        a = re.findall(r"\d{1,3}\sL",i.text,flags=re.I)
        if len(a) > 0:
            WATER_CAPACITY.append(a[0])
        else:
            WATER_CAPACITY.append(np.nan)

    # PRICE
    cost = soup.find_all('div', class_="hZ3P6w DeU9vF")[:PRODUCTS_PER_PAGE]
    for i in cost:
        PRICE.append(i.text.strip())

    # STARS
    data = soup.find_all('div', class_="RG5Slk")
    s = data[:PRODUCTS_PER_PAGE]

    for i in s:
        a = re.findall(r"\d\s*star", i.text, flags=re.I)
        STARS.append(a[0] if a else np.nan)

    # DOORS
    for i in data:
        v = re.findall(r"Single\sDoor|Double\sDoor", i.text, flags=re.I)
        DOORS.append(v[0] if v else np.nan)
```

```
AssertionError: Only 17 products found on page 12
```

INNOMATICS RESEARCH LABS

Summary of Raw Data

Data collected for 700+ refrigerator products data

Data issues observed:

- Warranty mixed with compressor details
- Inconsistent formats
- Missing values
- Duplicate entries

Unnamed: 0	BRAND	PRICE	DOORS	WATER_CAPACITY	STARS	WARRANTY	STABILIZER	COMPRESSOR_MODEL	COMPRESSOR_WARRANTY	PRICE_OFF	RATINGS
0	Whirlpool	₹15,790	Single Door	192 L	4 Star	1 Year Comprehensive Warranty	Built-in Stabilizer	Inverter Compressor	9 Years	28% off	4.4
1	Whirlpool	₹16,790	Single Door	184 L	5 Star	1 Year Comprehensive Warranty	Built-in Stabilizer	Inverter Compressor	9 Years	22% off	4.4
2	Samsung	₹13,990	Single Door	183 L	2 Star	1 Year Comprehensive Warranty	Built-in Stabilizer	NaN	10 Years	30% off	4.5
3	Samsung	₹16,990	Single Door	183 L	5 Star	1 year comprehensive warranty	Built-in Stabilizer	NaN	20 years	26% off	4.5
4	Haier	₹11,990	Single Door	185 L	2 Star	1 Year Warranty	Built-in Stabilizer	Normal Compressor	10 Years	29% off	4.3

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 719 entries, 0 to 718
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Unnamed: 0            719 non-null    int64
 1   BRAND                 719 non-null    object
 2   PRICE                 719 non-null    object
 3   DOORS                 651 non-null    object
 4   WATER_CAPACITY        719 non-null    object
 5   STARS                 667 non-null    object
 6   WARRANTY              544 non-null    object
 7   STABILIZER            700 non-null    object
 8   COMPRESSOR_MODEL      640 non-null    object
 9   COMPRESSOR_WARRANTY   578 non-null    object
 10  PRICE_OFF             719 non-null    object
 11  RATINGS               719 non-null    float64
dtypes: float64(1), int64(1), object(10)
memory usage: 67.5+ KB
```

INNOMATICS RESEARCH LABS

Data Cleaning Steps

- Removed symbols (₹, %, text units)
- Dropping duplicates
- Filling null values
- Extracting Product warranty
- Compressor warranty
- Converted columns to numeric
- Handled missing values and duplicates

- Removing unnessasary columns in table?

```
df.drop(columns="Unnamed: 0",inplace = True, axis=1)
```

```
df["PRICE_OFF"] = df['PRICE_OFF'].apply(lambda x: x.replace("%", "").split()[0])
```

```
df["WARRANTY"] = df["WARRANTY"].astype(str).apply(lambda x: " ".join(x.split()[:2]))
```

```
df["WARRANTY"]
```

```
0      1 Year
1      1 Year
2      1 Year
```

```
df["WARRANTY"].fillna('1 Year', inplace = True)
```

- filling null with mode ?

```
df["DOORS"] = df["DOORS"].fillna(df["DOORS"].mode()[0])
df["COMPRESSOR_MODEL"] = df["COMPRESSOR_MODEL"].fillna(df["COMPRESSOR_MODEL"].mode()[0])
df["STARS"] = df["STARS"].fillna(df["STARS"].mode()[0])
df["STABILIZER"] = df["STABILIZER"].fillna(df["STABILIZER"].mode()[0])
df["COMPRESSOR_WARRANTY"] = df["COMPRESSOR_WARRANTY"].fillna(df["COMPRESSOR_WARRANTY"].mode()[0])
```

- Replacing words and also converting data types like string to int

```
df["PRICE"] = df["PRICE"].apply(lambda x: int(x.replace("₹", "").replace(",", "")))
```

INNOMATICS RESEARCH LABS

Data Manipulation

- Change datatypes
- Created new structured columns:
  - WARRANTY_YEARS
  - COMPRESSOR_WARRANTY
- Converted text to numeric for analysis
- Grouped and aggregated data using Pandas

- Replacing words and also converting data types like string to int

```
df["PRICE"] = df["PRICE"].apply(lambda x: int(x.replace("₹", "").replace(",", "")))
```

```
df['PRICE'].astype(int)
```

```
0      15790
1      16790
```

```
df["PRICE_OFF"] = df['PRICE_OFF'].apply(lambda x: x.replace("%", "").split()[0])
```

```
df["PRICE_OFF"] = df['PRICE_OFF'].astype(int)
```

```
df.head()
```

	BRAND	PRICE	DOORS	WATER_CAPACITY	STARS	WARRANTY	STABILIZER	COMPRESSOR_MODEL	COMPRESSOR_WARRANTY	PRICE_OFF	RATINGS
0	Whirlpool	15790	Single Door	192	4 Star	1 Year	Built-in Stabilizer	Inverter Compressor	9 Years	28	4.4
1	Whirlpool	16790	Single Door	184	5 Star	1 Year	Built-in Stabilizer	Inverter Compressor	9 Years	22	4.4
2	Samsung	13990	Single Door	183	2 Star	1 Year	Built-in Stabilizer	Inverter Compressor	10 Years	30	4.5
3	Samsung	16990	Single Door	183	5 Star	1 year	Built-in Stabilizer	Inverter Compressor	20 years	26	4.5
4	Haier	11990	Single Door	185	2 Star	1 Year	Built-in Stabilizer	Normal Compressor	10 Years	29	4.3

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 719 entries, 0 to 718
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   BRAND                 719 non-null    object
 1   PRICE                 719 non-null    int64
 2   DOORS                 719 non-null    object
 3   WATER_CAPACITY        719 non-null    int64
 4   STARS                 719 non-null    object
 5   WARRANTY              719 non-null    object
 6   STABILIZER            719 non-null    object
 7   COMPRESSOR_MODEL      719 non-null    object
 8   COMPRESSOR_WARRANTY   719 non-null    object
 9   PRICE_OFF             719 non-null    int64
 10  RATINGS               719 non-null    float64
dtypes: float64(1), int64(3), object(7)
memory usage: 61.9+ KB
```

INNOMATICS RESEARCH LABS

Exploratory Data Analysis (EDA)

I used to practice with this dataset.

Filters

Group by

Pivot Table

```
df.PRICE.nsmallest(1)
```

```
469    8380
Name: PRICE, dtype: int64
```

- Find the price offer is 15% of the Godrej fridge ?

```
df[(df["BRAND"]=="Godrej") & (df["PRICE_OFF"]=="15% off")]
```

	BRAND	PRICE	DOORS	WATER_CAPACITY	STARS	WARRANTY	STABILIZER	COMPRESSOR_MODEL	COMPRESSOR_WARRANTY	PRICE_OFF	RATINGS
232	Godrej	₹69,990	Single Door	600	3 Star	1 Year	Built-in Stabilizer	Inverter Compressor	10 Years	15% off	4.2

- Find the 5 years of comperessor warrenty ?

```
df[df['COMPRESSOR_WARRANTY']== '5 Years']
```

	BRAND	PRICE	DOORS	WATER_CAPACITY	STARS	WARRANTY	STABILIZER	COMPRESSOR_MODEL	COMPRESSOR_WARRANTY	PRICE_OFF	RATINGS
175	Blue	₹8,990	Single Door	45	2 Star	1 Year	Built-in Stabilizer	Reciprocating Compressor	5 Years	20% off	4.1
469	BPL	₹8,380	Single Door	45	2 Star	1 Year	Built-in Stabilizer	Reciprocating Compressor	5 Years	32% off	4.2
677	Whirlpool	₹16,790	Single Door	184	5 Star	1 Year	Built-in Stabilizer	Metal Compressor Machine Room	5 Years	22% off	4.3

INNOMATICS RESEARCH LABS

- How would you select all refrigerators priced above ₹95,000 using loc?

```
df.loc[df['PRICE']>=95000,['BRAND','PRICE']]
```

	BRAND	PRICE
96	Haier	95490
151	Haier	95490
188	Haier	99490
267	Haier	95490
282	Samsung	105495
602	Haier	95490
613	Midea	99990
679	Samsung	99990

- which is the least price product in data frame.

```
df.loc[df['PRICE'].idxmin()]
```

```
BRAND                                       BPL
PRICE                                      8380
DOORS                                Single Door
WATER_CAPACITY                                45
STARS                                    2 Star
WARRANTY                                 1 Year
STABILIZER                   Built-in Stabilizer
COMPRESSOR_MODEL        Reciprocating Compressor
COMPRESSOR_WARRANTY                     5 Years
PRICE_OFF                                    32
RATINGS                                     4.2
Name: 469, dtype: object
```

- Using iloc, how do you extract: first 10 rows, only numeric columns?

```
df.iloc[:10,:2]
```

	BRAND	PRICE
0	Whirlpool	15790
1	Whirlpool	16790
2	Samsung	13990
3	Samsung	16990
4	Haier	11990
5	Samsung	15740
6	Voltas	12790
7	Whirlpool	12190
8	Samsung	25490
9	Haier	14490

INNOMATICS RESEARCH LABS

- We use group by for calculating average and sum of price for a particular brand.
- We used to calculate how many products particular brand by using size method.

- How do you calculate average price per brand?

```
df.groupby("BRAND")['PRICE'].mean()
```

```
BRAND
Acer           23999.000000
BOSCH          25616.375000
BPL             9350.000000
Blue            8990.000000
CANDY          28740.000000
Electrolux     86323.333333
Godrej         24574.843373
Haier          38061.591667
IFB            18906.666667
Kelvinator     13430.000000
Kenstar        12390.000000
LG             27821.820513
Liebherr       16990.000000
Lloyd          19125.400000
MOTOROLA       49815.000000
MarQ            8490.000000
Midea          33308.333333
ONIDA          12089.333333
Panasonic      29806.333333
Power          10519.000000
Rockwell       10200.000000
Samsung        32045.964912
Sharp           9916.000000
TOSHIBA        19900.000000
Voltas         26174.571429
Walton         18990.000000
Whirlpool      17738.575862
realme         19681.666667
Name: PRICE, dtype: float64
```

- Calculate total price per brand ?

```
df.groupby('BRAND')['PRICE'].sum()
```

```
BRAND
Acer            23999
BOSCH          409862
BPL             18700
Blue             8990
CANDY          172440
Electrolux     258970
Godrej        2039712
Haier         4567391
IFB            113440
Kelvinator      40290
Kenstar         24780
LG            1085051
Liebherr        16990
Lloyd          286881
MOTOROLA       199260
MarQ             8490
Midea          199850
ONIDA           36268
Panasonic      178838
Power           21038
Rockwell        10200
Samsung       1826620
Sharp           19832
TOSHIBA         19900
Voltas         916110
Walton          18990
Whirlpool     5144187
realme         236180
Name: PRICE, dtype: int64
```

- how many products are there in particular brand ?

```
df.groupby("BRAND").size()
```

```
BRAND
Acer            1
BOSCH          16
BPL             2
Blue            1
CANDY           6
Electrolux      3
Godrej         83
Haier         120
IFB             6
Kelvinator      3
Kenstar         2
LG             39
Liebherr        1
Lloyd          15
MOTOROLA        4
MarQ            1
Midea           6
ONIDA           3
Panasonic       6
Power           2
Rockwell        1
Samsung        57
Sharp           2
TOSHIBA         1
Voltas         35
Walton          1
Whirlpool     290
realme         12
dtype: int64
```

INNOMATICS RESEARCH LABS

- Whirlpool has the highest number of products and total revenue, showing strong market dominance, while Samsung and LG also contribute significantly to overall sales value.
- This third table shows the minimum and maximum price for each brand, calculated using Group By and aggregation functions.

```
df.groupby('BRAND').agg(
    no_of_pro = ('BRAND','count'),
    avg_price = ('PRICE','mean'),
    avg_water_l = ("WATER_CAPACITY", 'mean'),
    avg_discount = ('PRICE_OFF','mean'),
    total_per_brand = ("PRICE",'sum')
)
```

BRAND	no_of_pro	avg_price	avg_water_l	avg_discount	total_per_brand
Acer	1	23999.000000	215.000000	44.000000	23999
BOSCH	16	25616.375000	243.375000	34.750000	409862
BPL	2	9350.000000	65.000000	31.500000	18700
Blue	1	8990.000000	45.000000	20.000000	8990
CANDY	6	28740.000000	333.166667	29.500000	172440
Electrolux	3	86323.333333	479.000000	32.666667	258970
Godrej	83	24574.843373	245.530120	32.879518	2039712
Haier	120	38061.591667	348.850000	33.066667	4567391
IFB	6	18906.666667	208.833333	30.333333	113440
Kelvinator	3	13430.000000	92.333333	14.666667	40290
Kenstar	2	12390.000000	183.000000	34.000000	24780
LG	39	27821.820513	258.923077	26.333333	1085051
Liebherr	1	16990.000000	191.000000	24.000000	16990
Lloyd	15	19125.400000	199.533333	32.200000	286881
MOTOROLA	4	49815.000000	483.750000	41.500000	199260
MarQ	1	8490.000000	90.000000	43.000000	8490
Midea	6	33308.333333	266.666667	30.000000	199850
ONIDA	3	12089.333333	160.666667	35.666667	36268
Panasonic	6	29806.333333	251.666667	26.000000	178838
Power	2	10519.000000	69.000000	38.500000	21038
Rockwell	1	10200.000000	91.000000	20.000000	10200
Samsung	57	32045.964912	287.245614	27.964912	1826620
Sharp	2	9916.000000	71.000000	29.500000	19832
TOSHIBA	1	19900.000000	253.000000	27.000000	19900
Voltas	35	26174.571429	290.085714	40.371429	916110
Walton	1	18990.000000	190.000000	32.000000	18990
Whirlpool	290	17738.575862	201.506897	23.544828	5144187
realme	12	19681.666667	262.500000	47.666667	236180

```
df.pivot_table(
    index = 'BRAND',
    values = 'PRICE',
    aggfunc = 'sum'
)
```

BRAND	PRICE
Acer	23999
BOSCH	409862
BPL	18700
Blue	8990
CANDY	172440
Electrolux	258970
Godrej	2039712
Haier	4567391
IFB	113440
Kelvinator	40290
Kenstar	24780
LG	1085051
Liebherr	16990
Lloyd	286881
MOTOROLA	199260
MarQ	8490
Midea	199850
ONIDA	36268
Panasonic	178838
Power	21038
Rockwell	10200
Samsung	1826620
Sharp	19832
TOSHIBA	19900
Voltas	916110
Walton	18990
Whirlpool	5144187
realme	236180

```
df.groupby('BRAND').agg(
    min_pri_brand = ("PRICE","min"),
    max_pri_brand = ("PRICE","max"),
)
```

BRAND	min_pri_brand	max_pri_brand
Acer	23999	23999
BOSCH	17490	44652
BPL	8380	10320
Blue	8990	8990
CANDY	10990	67490
Electrolux	75990	92990
Godrej	10999	69999
Haier	11790	99490
IFB	13990	24490
Kelvinator	10350	19590
Kenstar	12390	12390
LG	10290	74990
Liebherr	16990	16990
Lloyd	10190	37999
MOTOROLA	13290	89990
MarQ	8490	8490
Midea	17900	99990
ONIDA	10390	12939
Panasonic	15999	42690
Power	9839	11199
Rockwell	10200	10200
Samsung	13290	105495
Sharp	9370	10462
TOSHIBA	19900	19900
Voltas	12290	67990
Walton	18990	18990
Whirlpool	12190	46350
realme	11590	45990

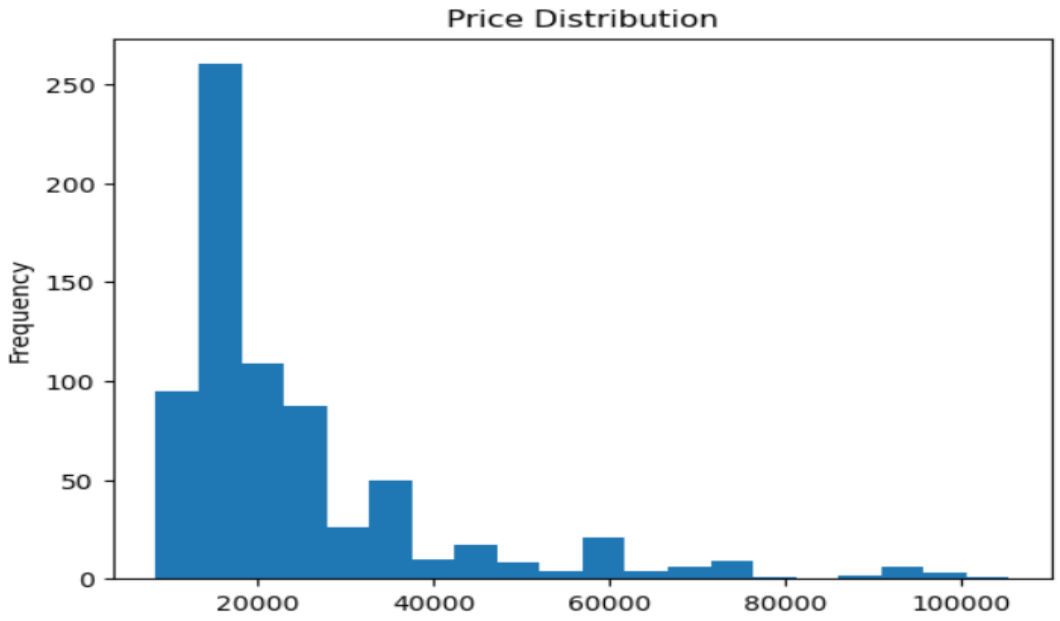
INNOMATICS RESEARCH LABS

VISUALIZATION

- Univariate Analysis
- About the Graph:
- Shows how product prices are distributed.
- Most products are priced in the lower range (around 15k–30k).
- Fewer products exist in higher price ranges.
- The distribution is right-skewed, meaning some expensive outliers increase the upper tail.

```
df['PRICE'].plot(kind='hist', bins=20, title='Price Distribution')
```

```
<Axes: title={'center': 'Price Distribution'}, ylabel='Frequency'>
```



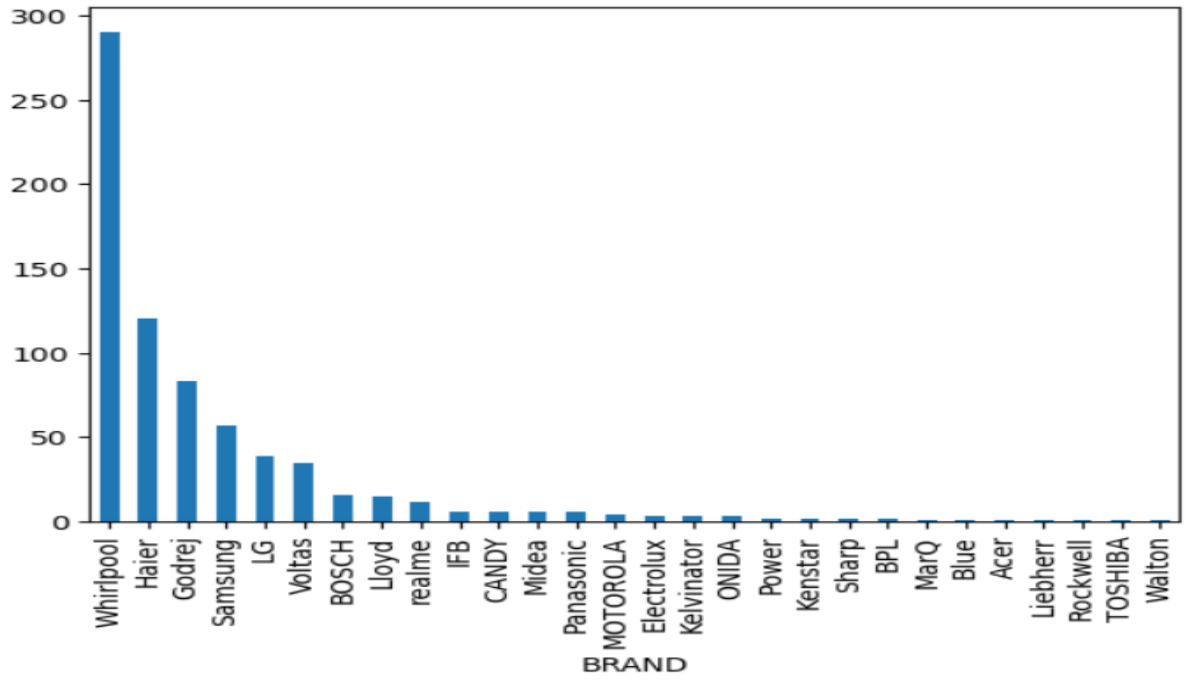
INNOMATICS RESEARCH LABS

About the Graph:

- Shows the number of products available per brand.
- One brand (e.g., Whirlpool) has the highest product count.
- A few brands have moderate listings (Haier, Godrej, Samsung, LG).
- Many brands have very few products (long tail).

```
df['BRAND'].value_counts().plot.bar()
```

```
<Axes: xlabel='BRAND'>
```



INNOMATICS RESEARCH LABS

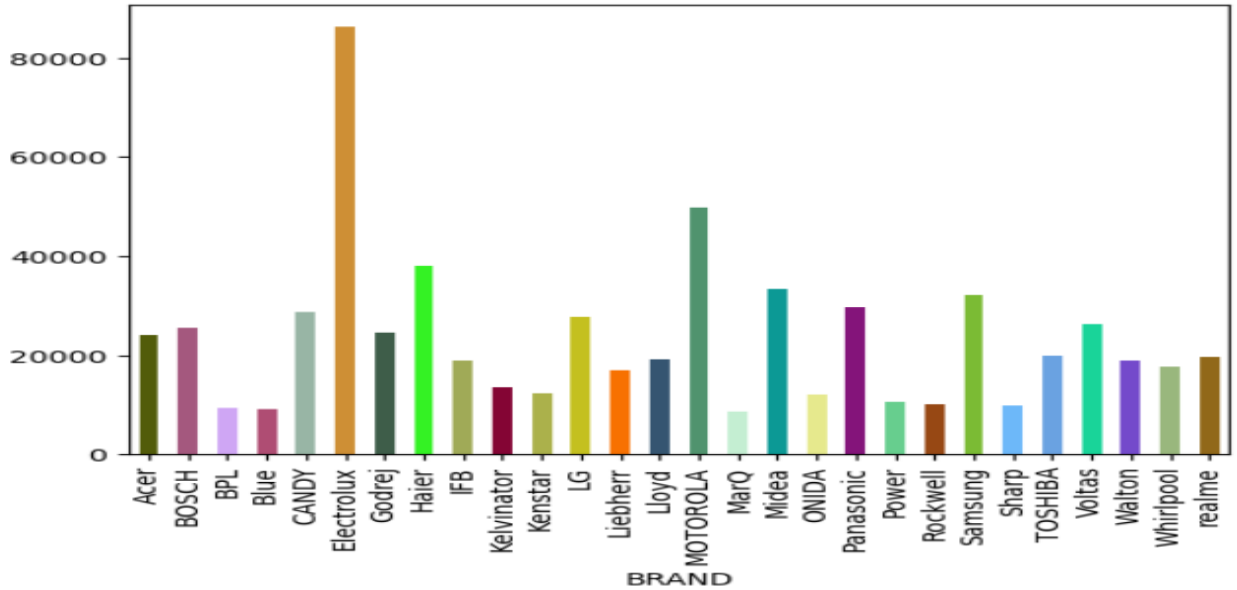
Bivariate Analysis

- Bivariate usually analyses two columns so we calculated average of price for each brand
- Shows average price comparison across brands.
- Some brands have significantly higher average prices (premium brands).
- Some brands offer lower average pricing (budget brands).
- Helps understand brand positioning in terms of pricing strategy.

```
data = df.groupby('BRAND')['PRICE'].mean()
colors = np.random.rand(len(data), 3)

data.plot.bar(color=colors)
```

```
<Axes: xlabel='BRAND'>
```

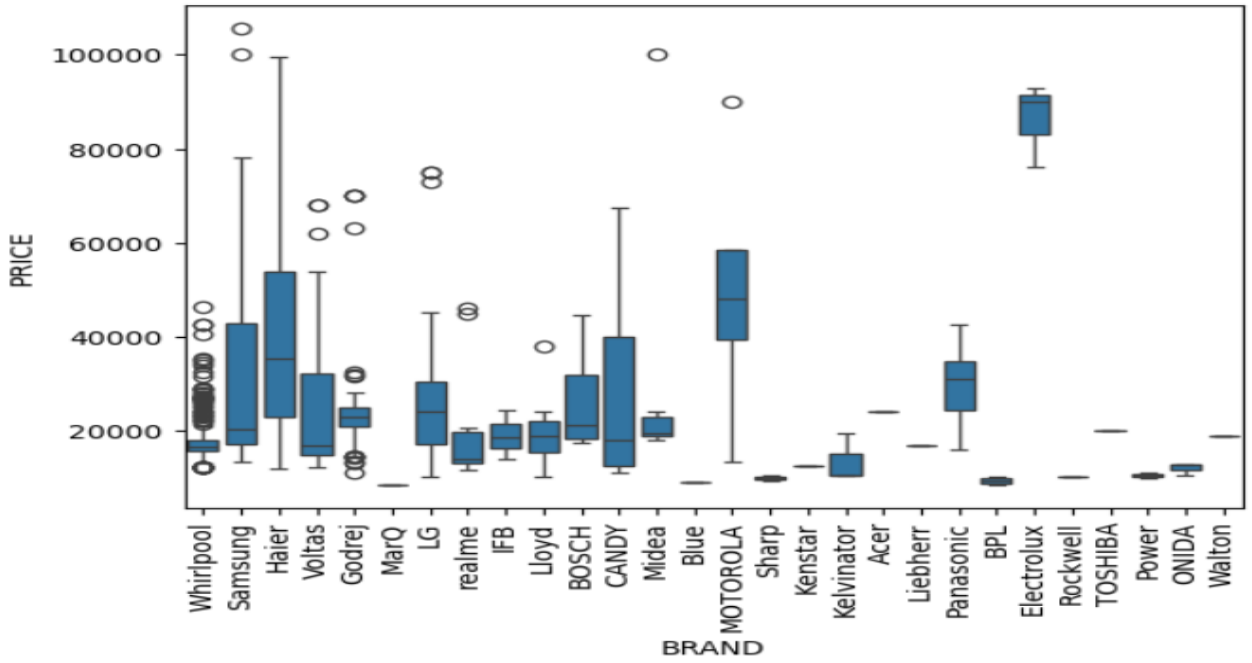


INNOMATICS RESEARCH LABS

What It Shows:

- Mostly boxplot shown outliers, Here boxplot is shown higher price.
- The middle line inside each box Median price.
- - The box → Middle 50% of prices (IQR).
- The whiskers Overall spread of data. The dots Outliers (very high or low prices).

```
plt.figsize=(10,16)
sns.boxplot(data=df, x= 'BRAND', y='PRICE')
plt.xticks(rotation=90)
plt.show()
```

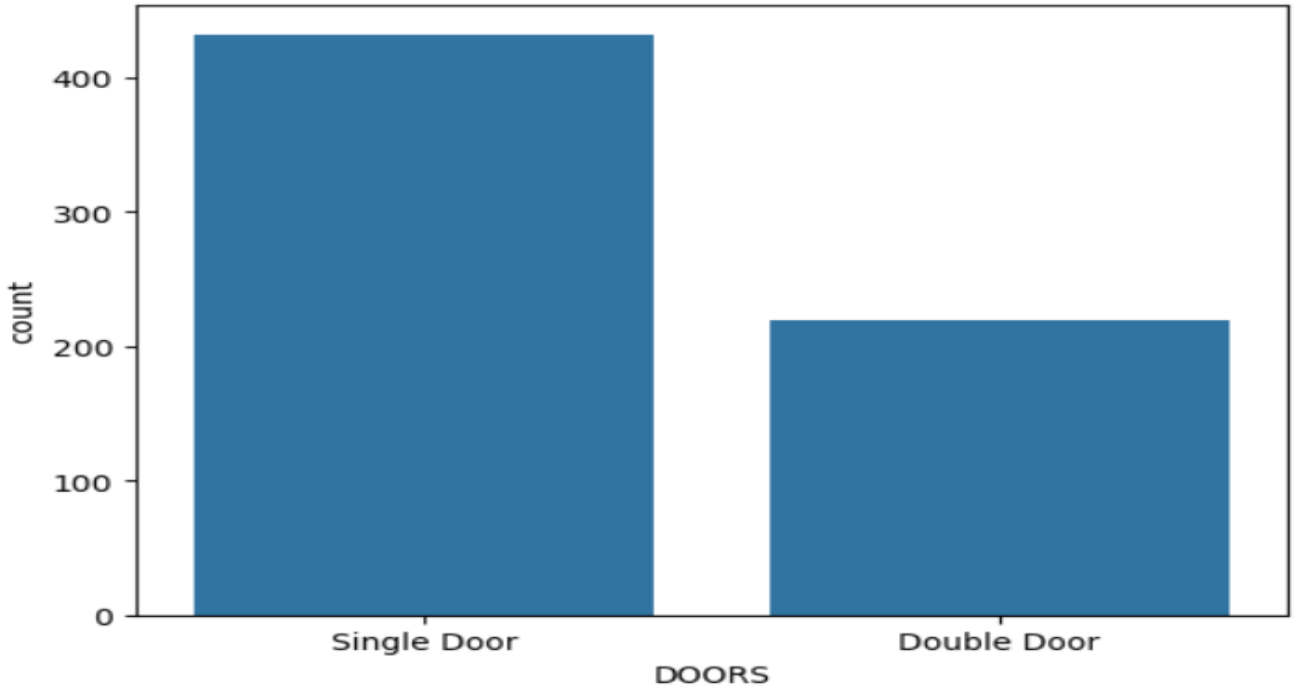


INNOMATICS RESEARCH LABS

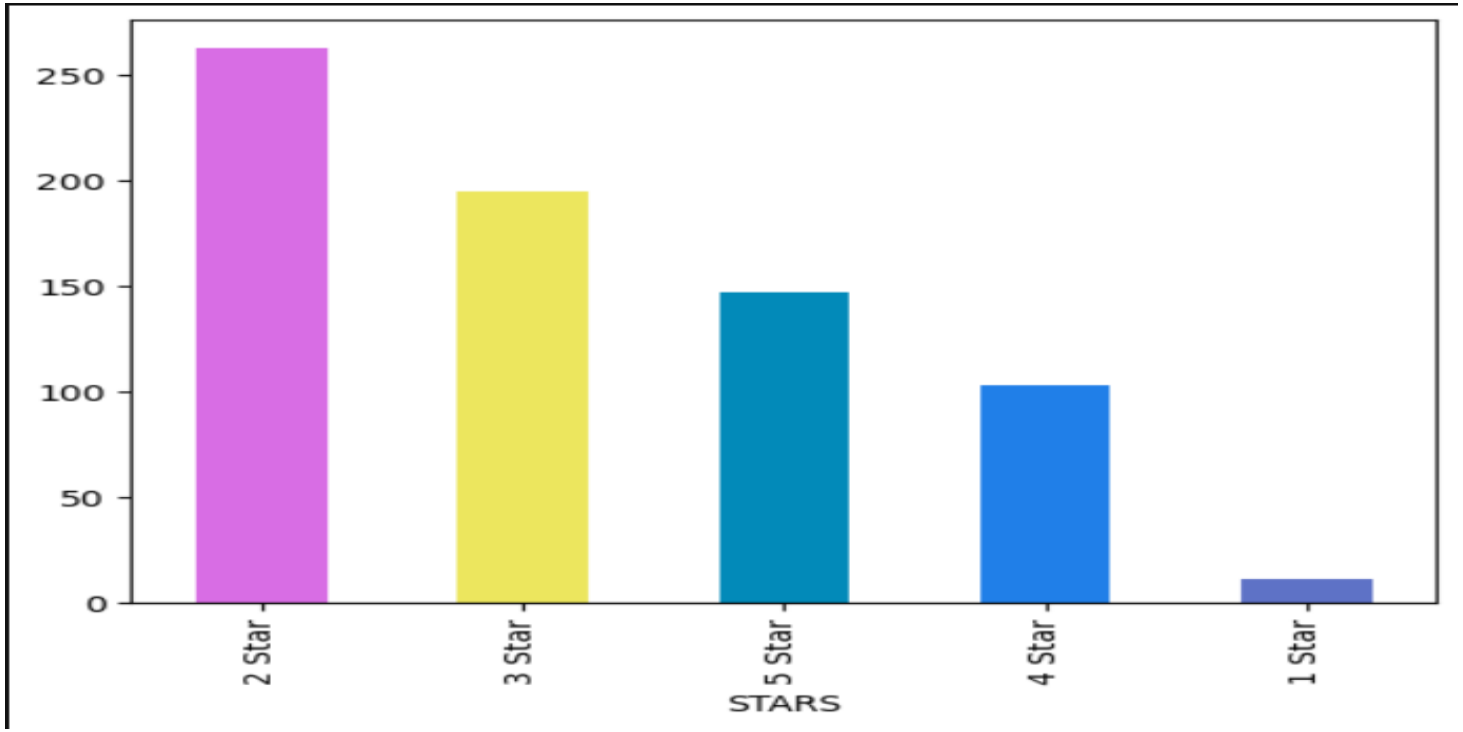
- About the Graph:
- Shows distribution of refrigerator types based on number of doors.
- Single Door products are significantly higher in number.
- Double Door products are fewer compared to single door.

```
sns.countplot(data=df, x='DOORS')
```

```
<Axes: xlabel='DOORS', ylabel='count'>
```



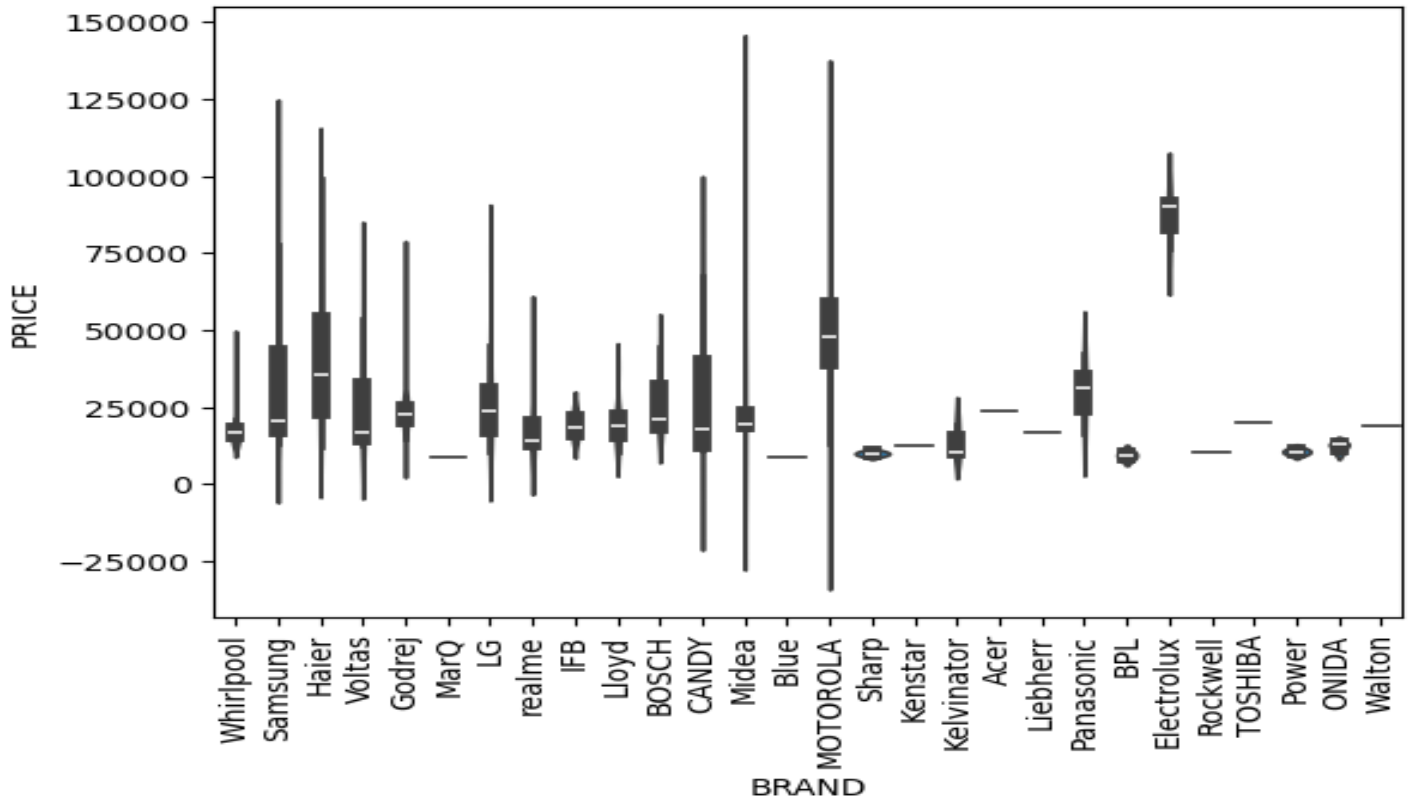
- About the Graph:¶
- Shows the distribution of product ratings (stars).
- 2-star ratings appear most frequently.
- 3-star and 5-star ratings follow next.
- 1-star ratings are very few.



INNOMATICS RESEARCH LABS

- This visualization represents a violin plot showing the distribution of refrigerator prices across different brands. It helps us understand how pricing varies between brands and how prices are distributed within each brand.

```
sns.violinplot(x='BRAND', y='PRICE', data=df)
plt.xticks(rotation=90);
```



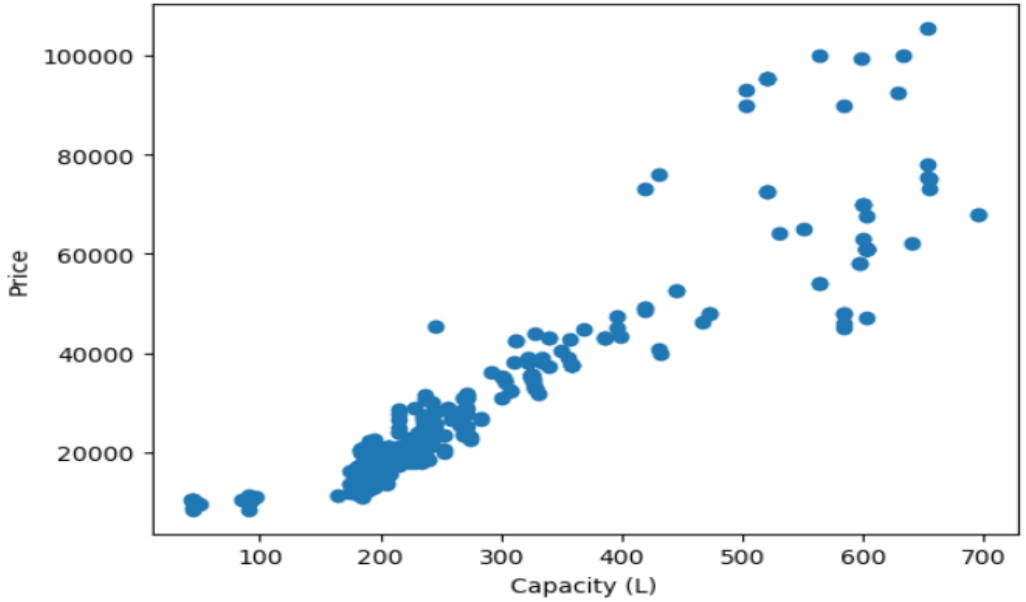
INNOMATICS
RESEARCH LABS

- # Multivariate Analysis

About Graph

- Shows the relationship between capacity and price.
- There is a positive correlation — as capacity increases, price generally increases.
- Higher-capacity products (500L+) tend to have higher prices. Some variation exists, but overall trend is upward.

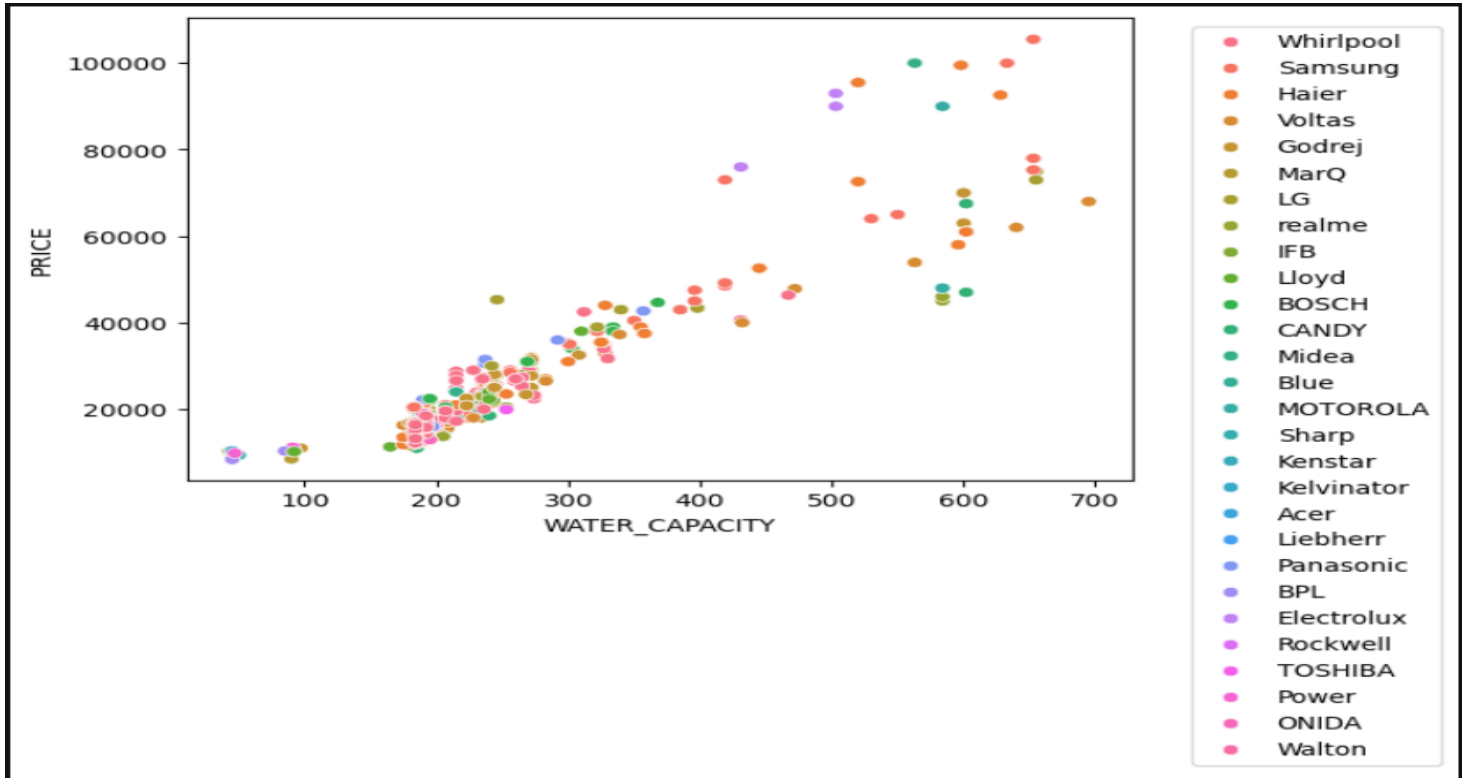
```
plt.scatter(df['WATER_CAPACITY'], df['PRICE'])
plt.xlabel('Capacity (L)')
plt.ylabel('Price')
plt.show()
```



INNOMATICS RESEARCH LABS

About Graph¶

- Shows the relationship between capacity and price for each brand.
- There is a positive trend — higher capacity generally means higher price.
- Different colors help compare how brands price their products.
- Some brands have premium pricing even at similar capacities.



INNOMATICS RESEARCH LABS

CONCLUTION

- **Simple summary:**

- Built an end-to-end web scraping and data analysis pipeline to extract refrigerator product data from an e-commerce website.
- Transformed unstructured HTML data into a clean, structured dataset using Python, Pandas, and regex.
- Conducted exploratory data analysis and created visualizations to compare pricing, capacity, rating, warranty, and product features across brands.

THANK YOU

INNOMATICS
RESEARCH LABS